

# SQL JOINS

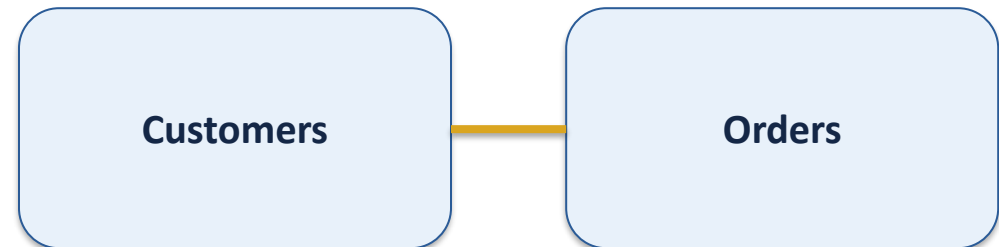
Combining Related Data from Multiple Tables

**Database Fundamentals**

# What Is a SQL JOIN?

The basic idea behind combining related records

- A JOIN combines rows from two or more tables into a single query result.
- The rows are normally matched using a related column, such as a primary key and foreign key.
- JOINS let us keep data in separate, well-structured tables while still retrieving useful combined information.
- For example, customer details can be stored in Customers while purchase details are stored in Orders.



**Related rows are combined**

# Why Do We Need JOINS?

Relational databases normally divide information into related tables

- Storing everything in one large table creates repeated data and makes updates harder.
- A normalized database separates entities such as customers, orders, products, and employees.
- JOINS allow a query to reconstruct the information needed by an application or report.
- The JOIN condition tells SQL which records belong together.

| Customers | Orders             |
|-----------|--------------------|
| 1 — Ved   | 101 — 1 — Laptop   |
| 2 — Aisha | 102 — 3 — Mouse    |
| 3 — Rahul | 103 — 1 — Keyboard |

```
ON Customers.customer_id = Orders.customer_id
```

# Sample Database

We will use the same tables throughout the presentation

| customer_id | customer_name | city      |
|-------------|---------------|-----------|
| 1           | Ved           | Vadodara  |
| 2           | Aisha         | Ahmedabad |
| 3           | Rahul         | Surat     |
| 4           | Meera         | Rajkot    |

| order_id | customer_id | product  | amount  |
|----------|-------------|----------|---------|
| 101      | 1           | Laptop   | ₹60,000 |
| 102      | 1           | Mouse    | ₹1,200  |
| 103      | 3           | Keyboard | ₹2,500  |
| 104      | 5           | Monitor  | ₹15,000 |

**Notice: Meera has no order, while Order 104 refers to customer\_id 5, which is not present in Customers.**

# INNER JOIN

Returns only rows that have a match in both tables

- INNER JOIN keeps the matching records from both tables.
- Customers without orders are excluded from the result.
- Orders whose customer does not exist in Customers are also excluded.
- It is the most common JOIN when we only want related records.

```
SELECT c.customer_name, o.product, o.amount
FROM Customers c
INNER JOIN Orders o
    ON c.customer_id = o.customer_id;
```

| customer_name | product  | amount  |
|---------------|----------|---------|
| Ved           | Laptop   | ₹60,000 |
| Ved           | Mouse    | ₹1,200  |
| Rahul         | Keyboard | ₹2,500  |

# LEFT JOIN

Keeps every row from the left table

- LEFT JOIN keeps every customer, even when that customer has no order.
- When there is no matching order, columns from Orders contain NULL.
- This is useful when we want a complete list from the left table.
- It is commonly used to find records that do not have a related record.

```
SELECT c.customer_name, o.product, o.amount
FROM Customers c
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id;
```

| customer_name | product  | amount  |
|---------------|----------|---------|
| Ved           | Laptop   | ₹60,000 |
| Ved           | Mouse    | ₹1,200  |
| Rahul         | Keyboard | ₹2,500  |
| Aisha         | NULL     | NULL    |
| Meera         | NULL     | NULL    |

# RIGHT JOIN

Keeps every row from the right table

- RIGHT JOIN keeps every order, even if no matching customer exists.
- When the customer is missing, columns from Customers contain NULL.
- It is the mirror image of LEFT JOIN with the table order reversed.
- Many developers prefer LEFT JOIN because it is often easier to read.

```
SELECT c.customer_name, o.order_id, o.product
FROM Customers c
RIGHT JOIN Orders o
    ON c.customer_id = o.customer_id;
```

| customer_name | order_id | product  |
|---------------|----------|----------|
| Ved           | 101      | Laptop   |
| Ved           | 102      | Mouse    |
| Rahul         | 103      | Keyboard |
| NULL          | 104      | Monitor  |

# FULL OUTER JOIN

Keeps all rows from both tables

- FULL OUTER JOIN keeps matching rows as well as unmatched rows from both sides.
- If a row exists only in Customers, Order columns become NULL.
- If a row exists only in Orders, Customer columns become NULL.
- Some database systems, including MySQL, do not support FULL OUTER JOIN directly.

```
SELECT c.customer_name, o.order_id, o.product
FROM Customers c
FULL OUTER JOIN Orders o
    ON c.customer_id = o.customer_id;
```

| customer_name | order_id | product  |
|---------------|----------|----------|
| Ved           | 101      | Laptop   |
| Ved           | 102      | Mouse    |
| Rahul         | 103      | Keyboard |
| Aisha         | NULL     | NULL     |
| Meera         | NULL     | NULL     |
| NULL          | 104      | Monitor  |

# CROSS JOIN

Creates every possible combination

- CROSS JOIN does not need an ON condition.
- Every row from the first table is paired with every row from the second table.
- If table A has 4 rows and table B has 3 rows, the result contains 12 rows.
- It is useful for generating combinations, but large tables can create very large results.

```
SELECT c.customer_name, p.product_name
FROM Customers c
CROSS JOIN Products p;
```

| Customer | Product  |
|----------|----------|
| Ved      | Laptop   |
| Ved      | Mouse    |
| Ved      | Keyboard |
| Aisha    | Laptop   |
| Aisha    | Mouse    |
| Aisha    | Keyboard |

# SELF JOIN

Joining a table to itself

- A SELF JOIN uses the same table twice in one query.
- Table aliases are important because SQL needs to distinguish the two copies.
- A common example is an Employees table containing both employee\_id and manager\_id.
- The JOIN connects an employee's manager\_id to another employee's employee\_id.

| employee_id | employee_name | manager_id |
|-------------|---------------|------------|
| 1           | Anita         | NULL       |
| 2           | Rahul         | 1          |
| 3           | Meera         | 1          |
| 4           | Dev           | 2          |

```
SELECT e.employee_name,  
       m.employee_name AS manager  
FROM Employees e  
LEFT JOIN Employees m  
  ON e.manager_id = m.employee_id;
```

# ON vs WHERE

Two clauses with different jobs

- ON describes how rows from the tables are related and matched.
- WHERE filters rows after the FROM/JOIN result has been formed.
- With an OUTER JOIN, moving a condition from ON to WHERE can change which rows survive.
- A good habit is to put relationship conditions in ON and result filters in WHERE.

```
-- Relationship
ON c.customer_id = o.customer_id

-- Filter
WHERE o.amount > 5000;
```

| Clause | Main purpose                             |
|--------|------------------------------------------|
| ON     | Defines the relationship / matching rule |
| WHERE  | Filters the resulting rows               |

# JOIN Types — Quick Comparison

Choose the JOIN based on which rows you need to retain

| JOIN       | Rows retained        | Typical use                                   |
|------------|----------------------|-----------------------------------------------|
| INNER      | Only matching rows   | Records existing in both tables               |
| LEFT       | All left + matches   | All customers, including those without orders |
| RIGHT      | All right + matches  | All orders, including unmatched orders        |
| FULL OUTER | Everything from both | Complete comparison of two tables             |
| CROSS      | Every combination    | Generate combinations                         |
| SELF       | Same table twice     | Employee-manager hierarchies                  |

# Practice Queries

Try writing the SQL before checking the solution

- Find all customers who have never placed an order.
- Display every order together with the customer's name.
- Display every customer and their orders, including customers with no orders.
- Find orders whose customer does not exist in the Customers table.
- Generate every possible customer-product combination.

Hint:

1. LEFT JOIN + WHERE ... IS NULL
2. INNER JOIN
3. LEFT JOIN
4. RIGHT/FULL JOIN

# Key Takeaways

- ✓ A JOIN combines related data from multiple tables.
- ✓ INNER JOIN keeps only matches.
- ✓ LEFT and RIGHT JOIN preserve all rows from one side.
- ✓ FULL OUTER JOIN preserves rows from both sides.
- ✓ CROSS JOIN creates every possible combination.
- ✓ SELF JOIN uses the same table twice.
- ✓ Use ON for relationships and WHERE for filtering.